

Design and Analysis of Algorithms

Sorting in Linear Time

Counting Sort



Counting Sort

Counting sort is a way to arrange the items in a vector by counting how many times each item appears. We then put the elements in order based on those counts.

Here's how the algorithm works

- Create a counting vector whose size is one more than the largest element in the unsorted vector.
- Fill each index of the counting vector with the number of occurrences of the corresponding value in the unsorted vector.
- Create a sorted vector using the value of each index in the counting vector.



Counting Sort

- Counting sort assumes that each of the n *input elements* is an integer in the *range 0 to k*. that is n is the number of elements and k is the highest value element.
- Consider the input set : 4, 1, 3, 4, 3. Then $n=5$ and $k=4$
- Counting sort determines for each input element x , the number of elements less than x . And it uses this information to place element x directly into its position in the output array. For example if there exists 17 elements less than x then x is placed into the 18th position into the output array.

Counting Sort

- **Input Array:** $A[1..n]$ store input data where $A[j] \in \{1, 2, 3, \dots, k\}$
- **Output Array:** $B[1..n]$ finally store the sorted data
- **Temporary Array:** $C[1..k]$ store data temporarily

Counting Sort

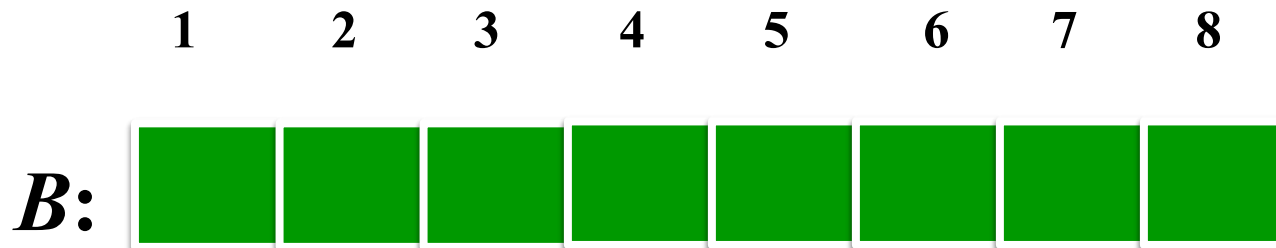
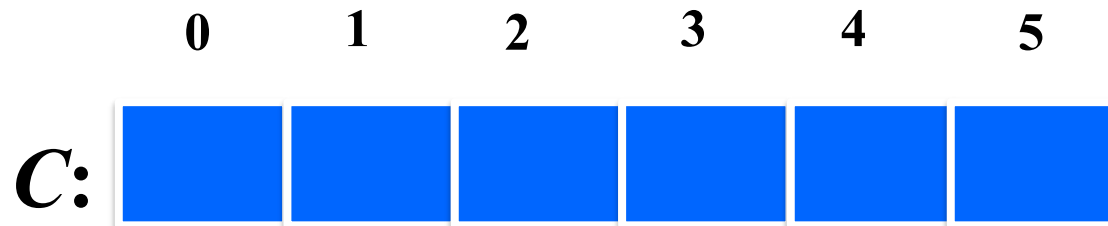
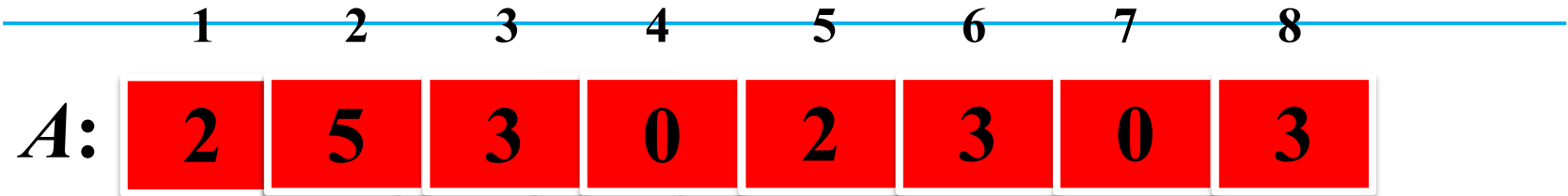
1. Counting-Sort(A, B, k)
2. Let $C[0 \dots k]$ be a new array
3. for $i=0$ to k
4. $C[i] = 0;$
5. for $j=1$ to A.length or n
6. $C[A[j]] = C[A[j]] + 1;$
7. for $i=1$ to k
8. $C[i] = C[i] + C[i-1];$
9. for $j=n$ or A.length down to 1
10. $B[C[A[j]]] = A[j];$
11. $C[A[j]] = C[A[j]] - 1;$

Counting Sort

1. Counting-Sort(A, B, k)
2. Let $C[0 \dots k]$ be a new array
3. for $i=0$ to k **[Loop 1]**
4. $C[i] = 0;$
5. for $j=1$ to $A.length$ (or n) **[Loop 2]**
6. $C[A[j]] = C[A[j]] + 1;$
7. for $i=1$ to k **[Loop 3]**
8. $C[i] = C[i] + C[i-1];$
9. for $j=n$ or $A.length$ down to 1 **[Loop 4]**
10. $B[C[A[j]]] = A[j];$
11. $C[A[j]] = C[A[j]] - 1;$

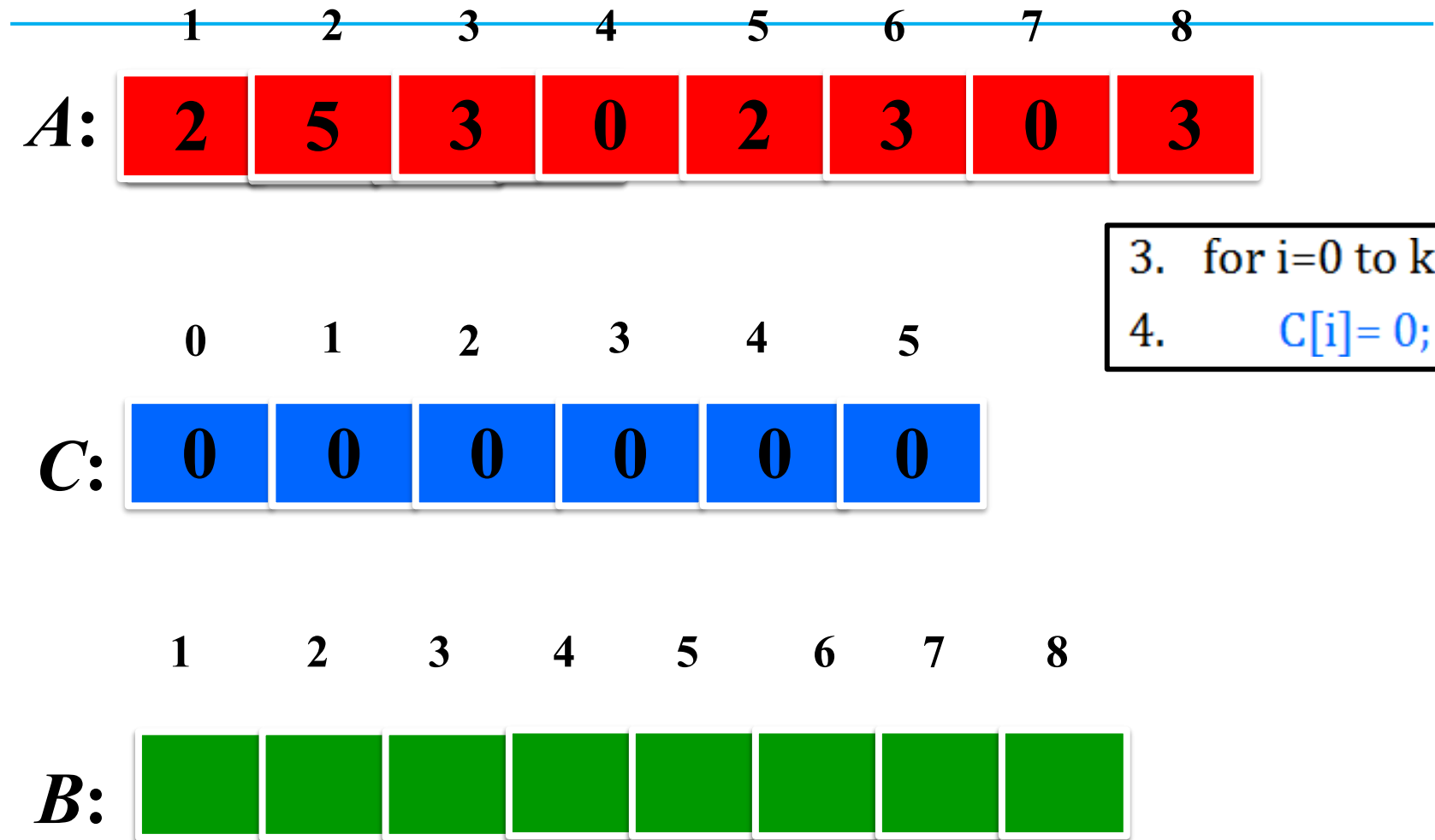


Counting-sort example

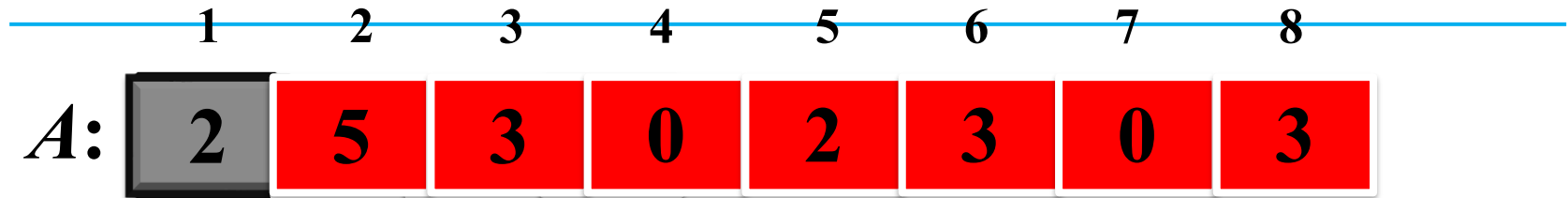




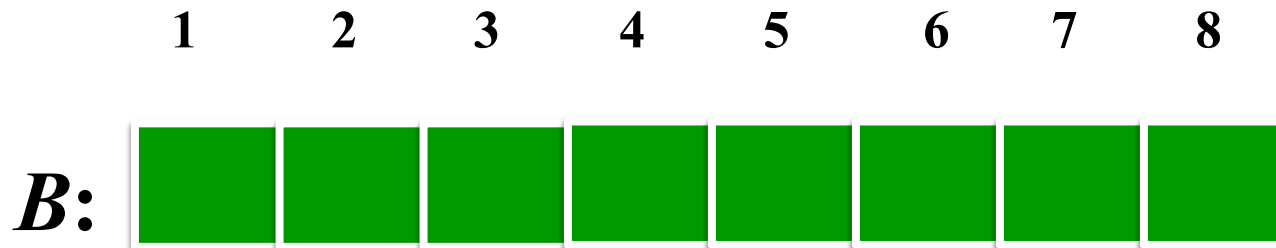
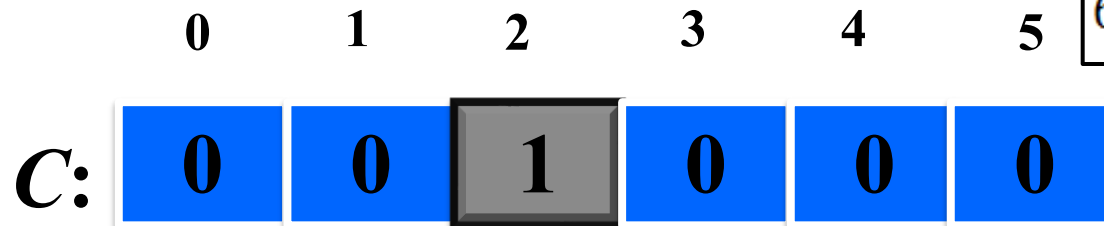
Executing Loop 1



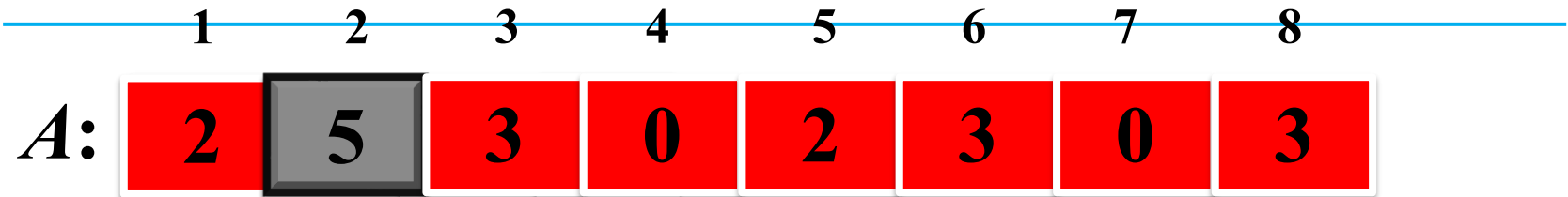
Executing Loop 2



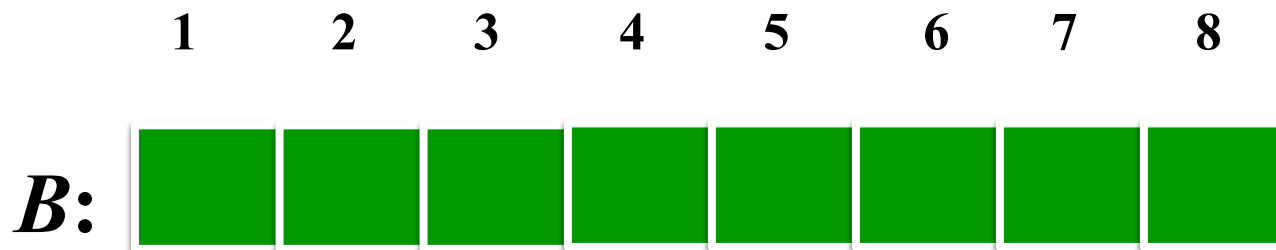
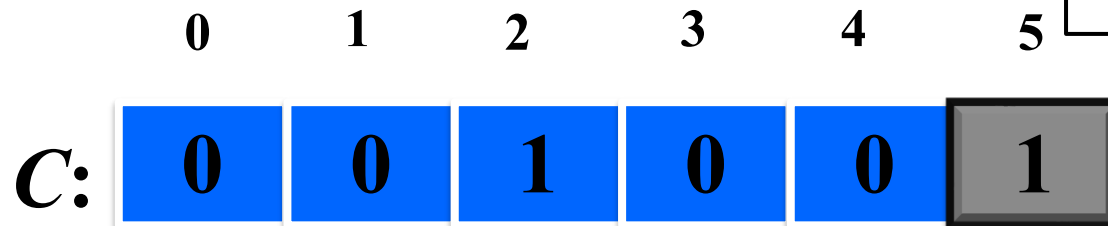
5. for j=1 to A.length or n
6. $C[A[j]] = C[A[j]] + 1;$



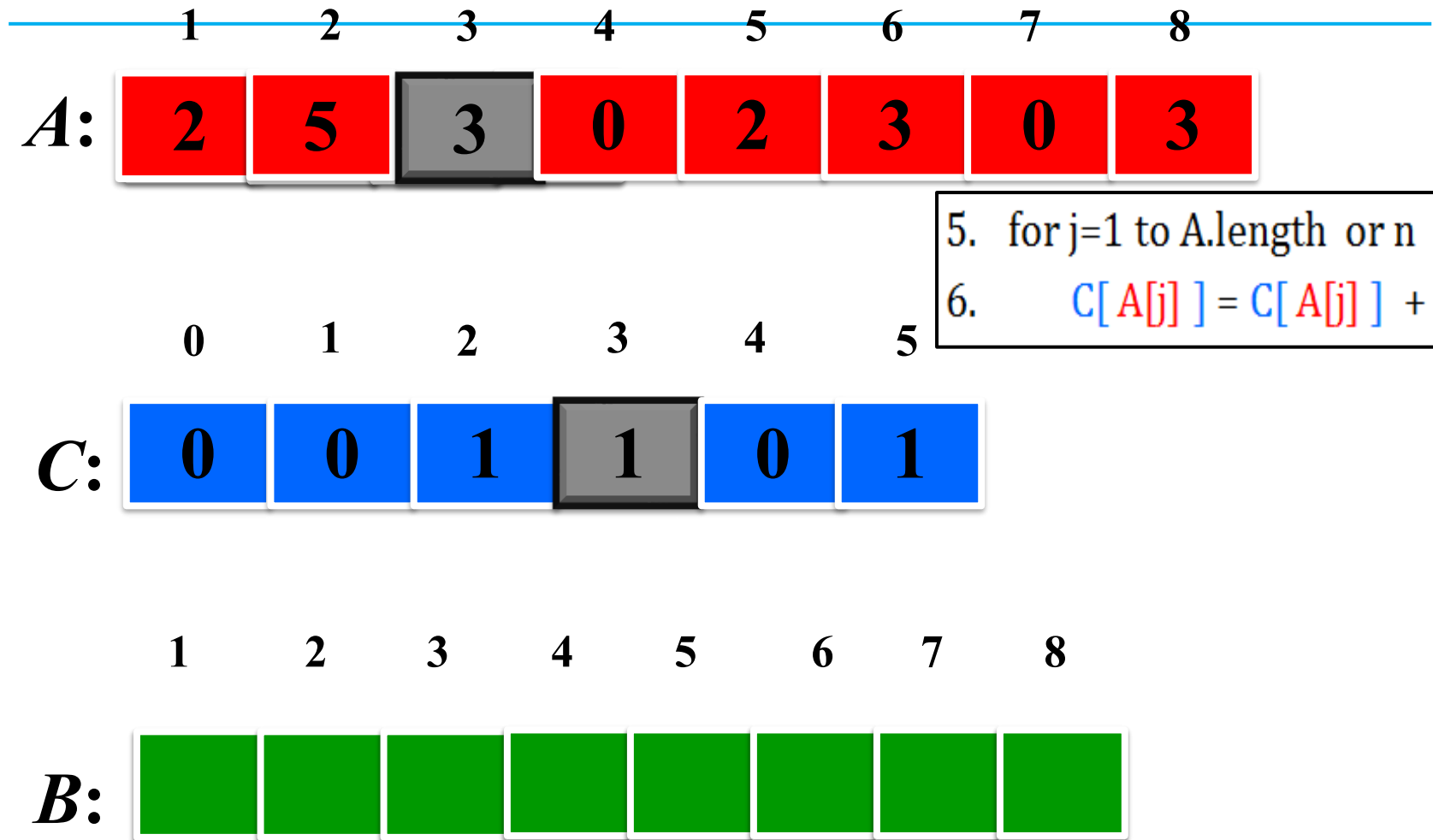
Executing Loop 2



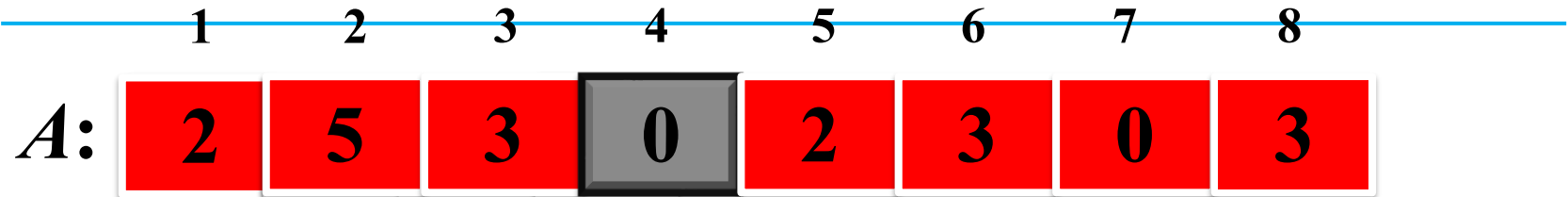
5. for j=1 to A.length or n
6. $C[A[j]] = C[A[j]] + 1;$



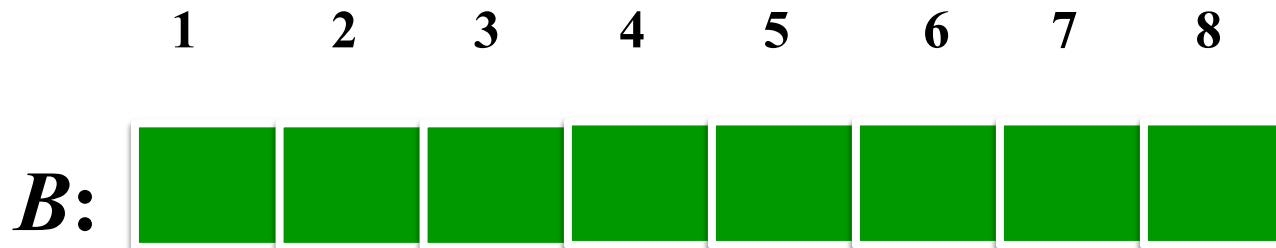
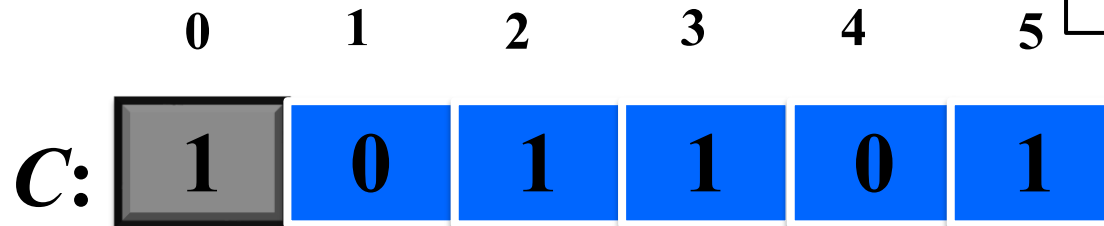
Executing Loop 2



Executing Loop 2

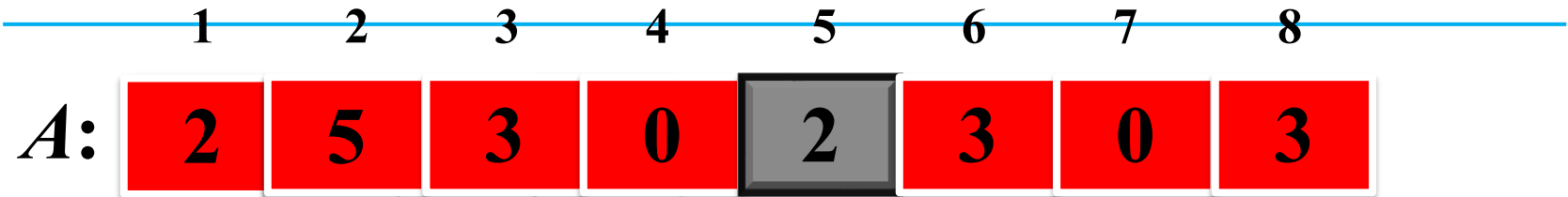


5. for j=1 to A.length or n
6. $C[A[j]] = C[A[j]] + 1;$

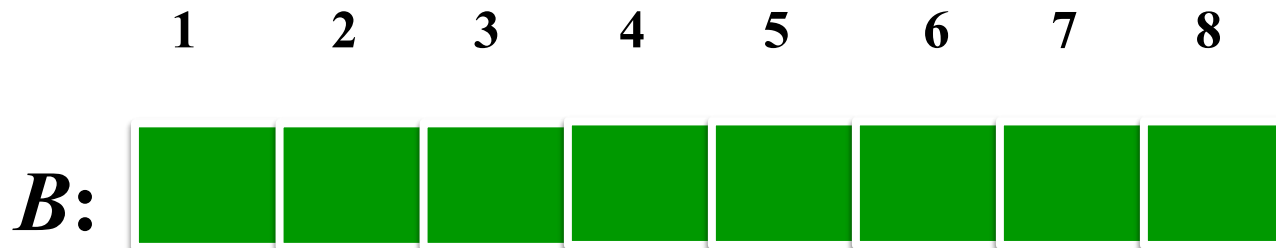
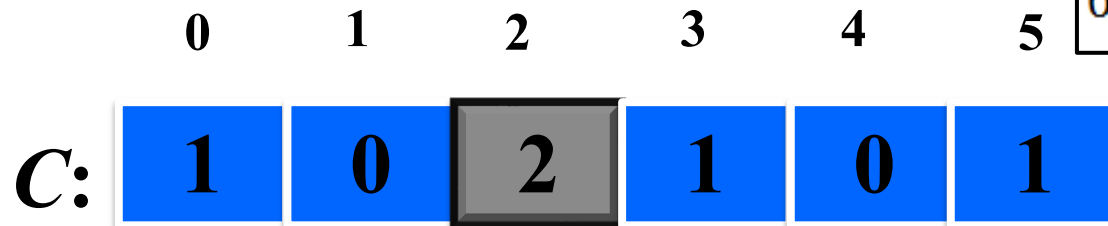




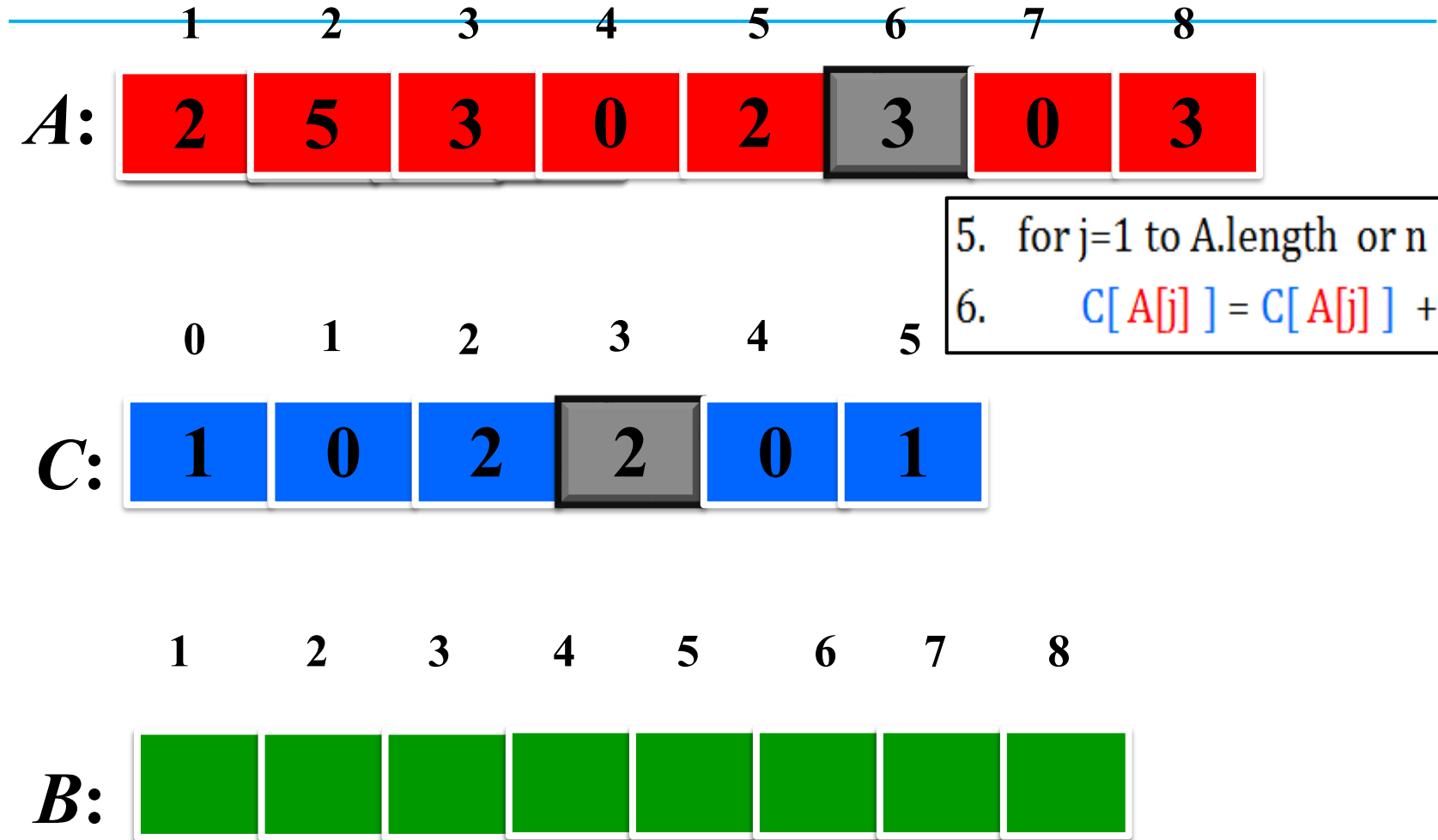
Executing Loop 2



5. for j=1 to A.length or n
6. $C[A[j]] = C[A[j]] + 1;$

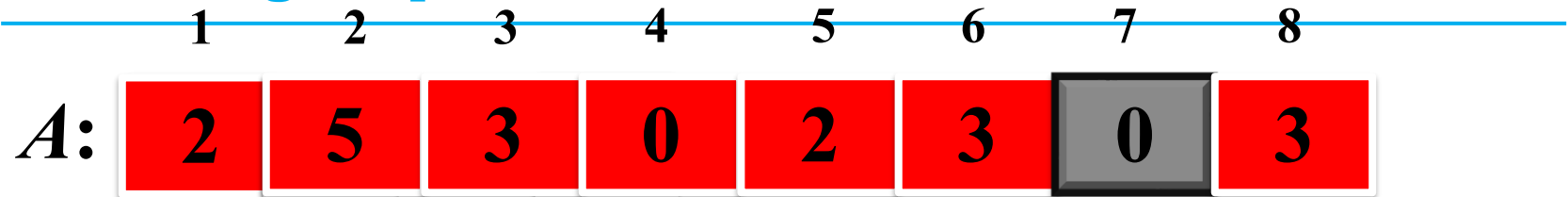


Executing Loop 2



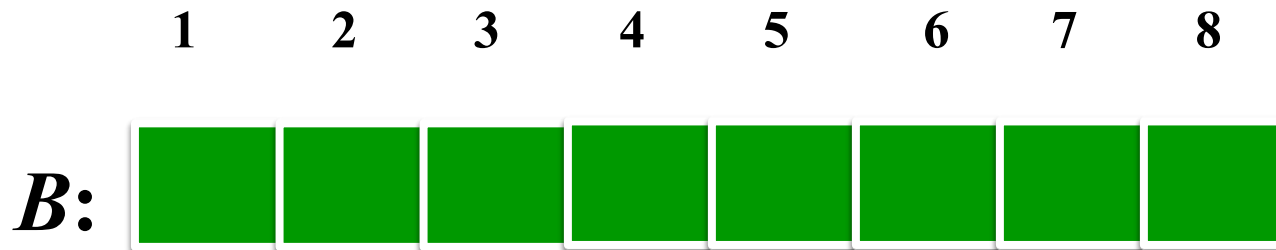
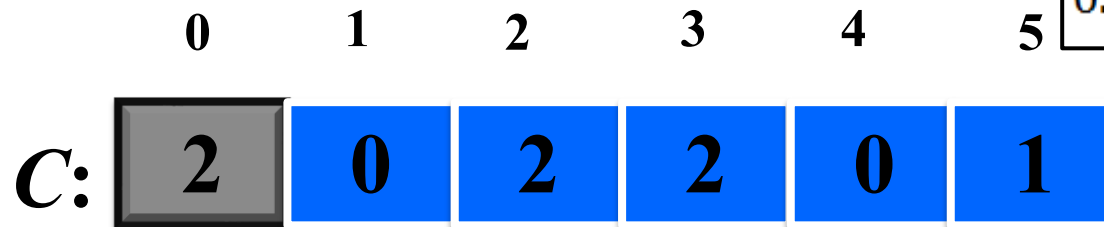


Executing Loop 2



5. for j=1 to A.length or n

6. $C[A[j]] = C[A[j]] + 1;$



A horizontal number line with tick marks labeled 1 through 8.

A horizontal array of eight boxes. The first seven boxes are red and contain the numbers 2, 5, 3, 0, 2, 3, and 0 respectively. The eighth box is gray and contains the number 3.

```

5.  for j=1 to A.length or n
6.      C[ A[j] ] = C[ A[j] ] + 1;

```

0	1	2	3	4	5	6
2	0	2	3	0	1	

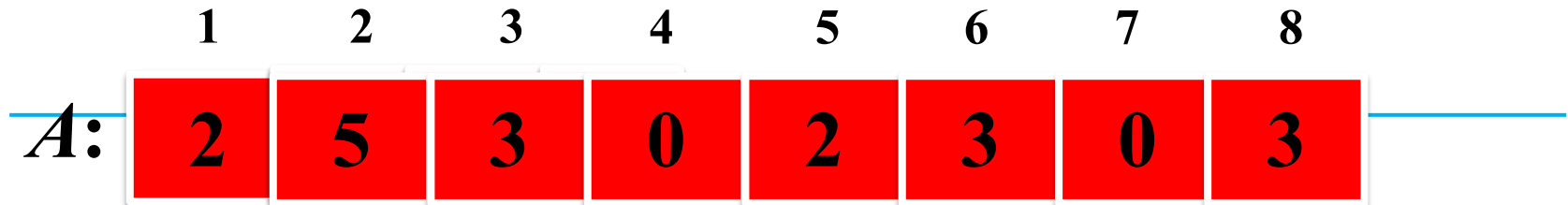
1	2	3	4	5	6	7	8

Diagram illustrating the state of three arrays A , B , and C during the execution of the algorithm.

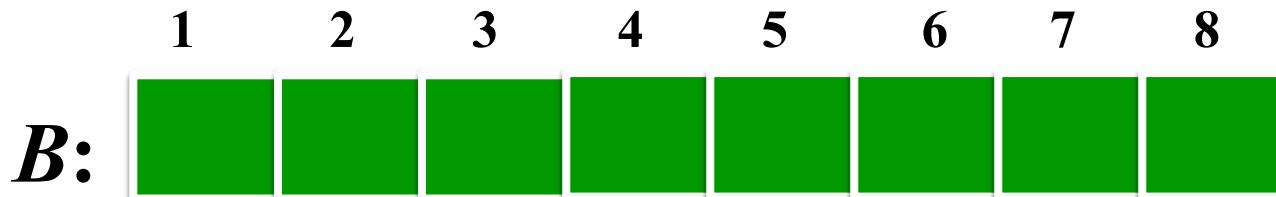
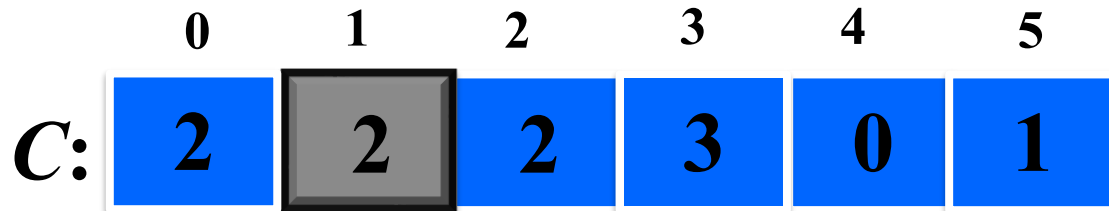
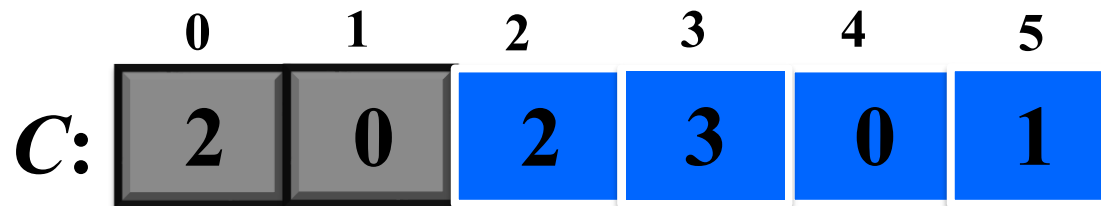
- Array A (red boxes) contains values: 2, 5, 3, 0, 2, 3, 0, 3. Indices 1 through 8 are shown above the boxes.
- Array B (green boxes) is empty. Indices 1 through 8 are shown above the boxes.
- Array C (blue boxes) contains values: 2, 0, 2, 3, 0, 1. Indices 0 through 5 are shown above the boxes.



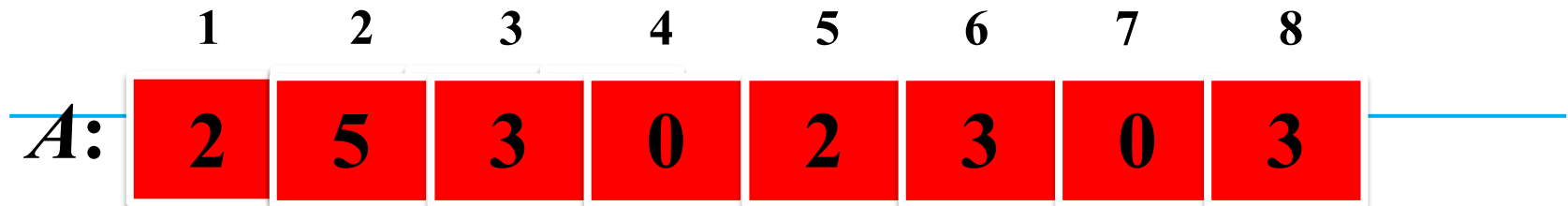
Executing Loop 3



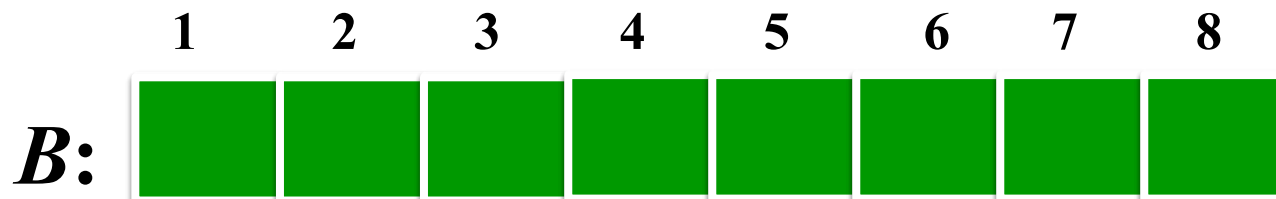
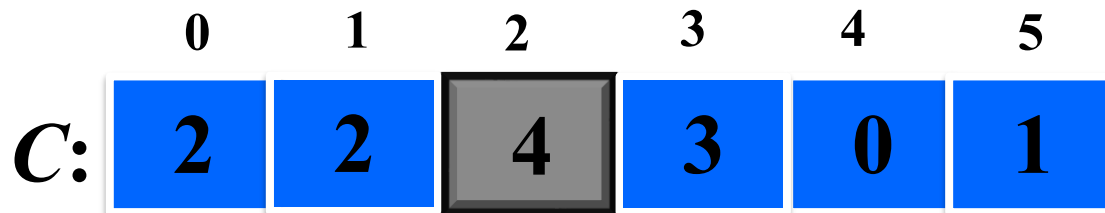
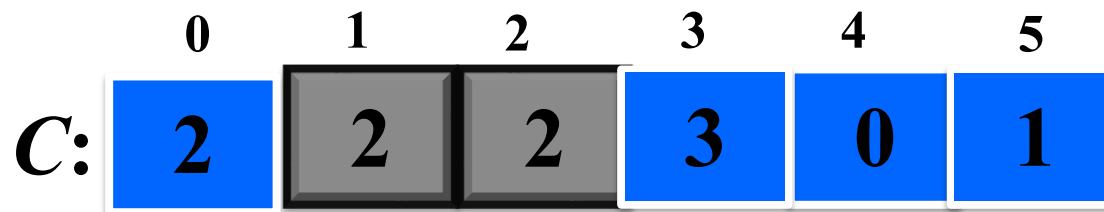
7. for i=1 to k
8. $C[i] = C[i] + C[i-1];$



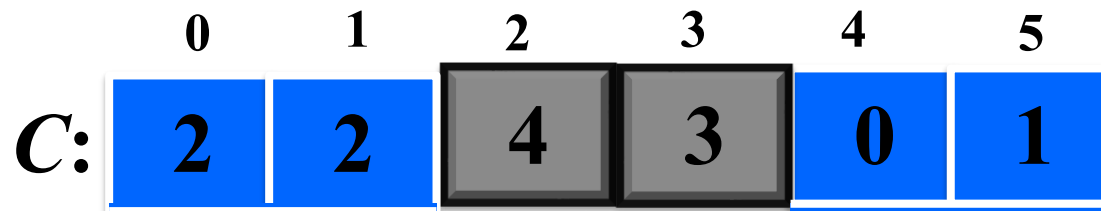
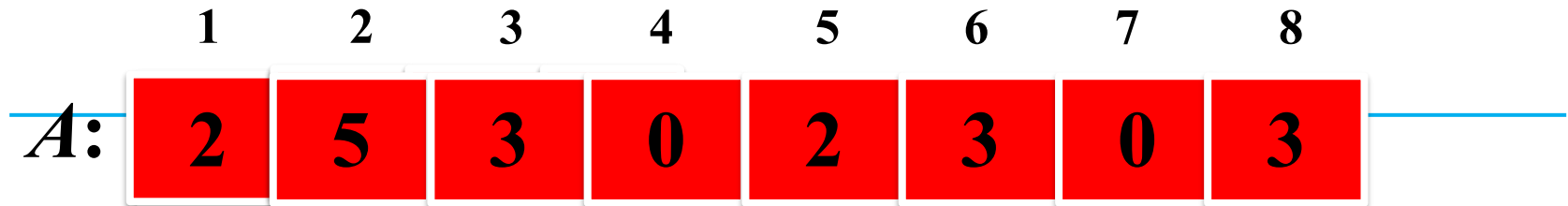
Executing Loop 3



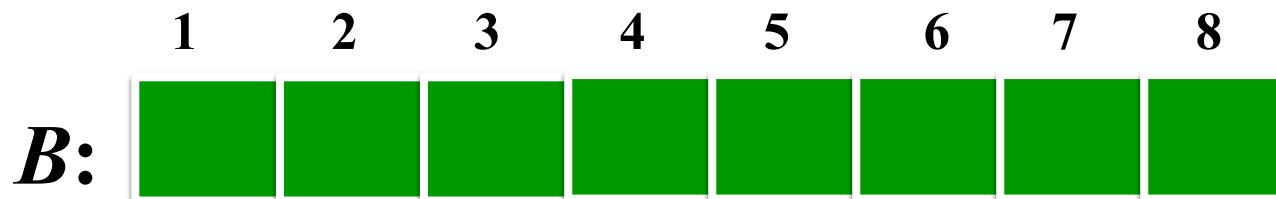
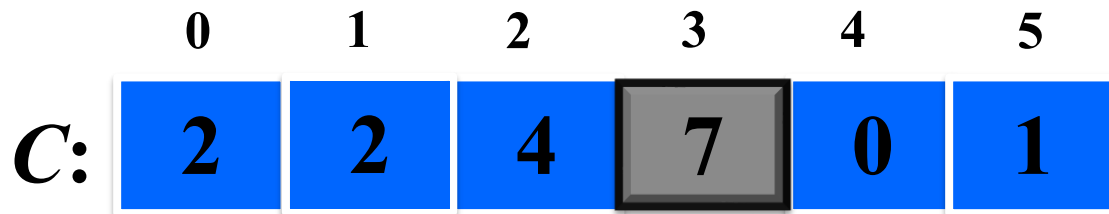
7. for i=1 to k
8. $C[i] = C[i] + C[i-1];$



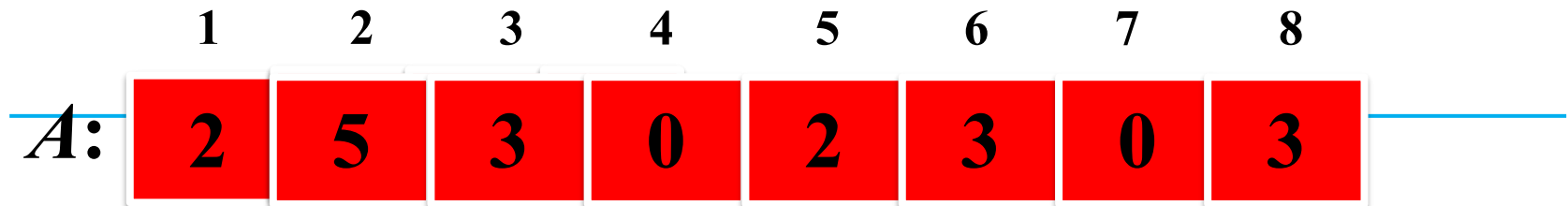
Executing Loop 3



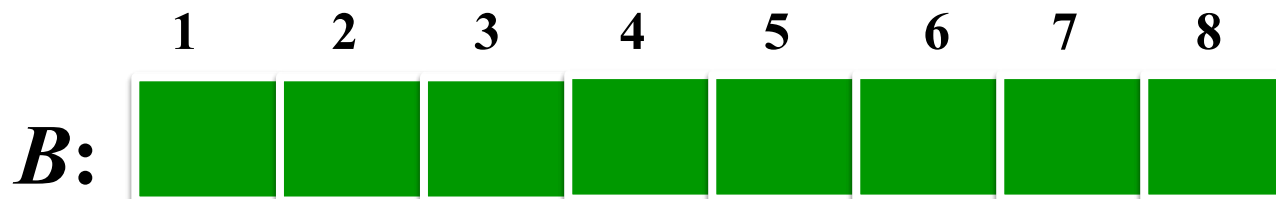
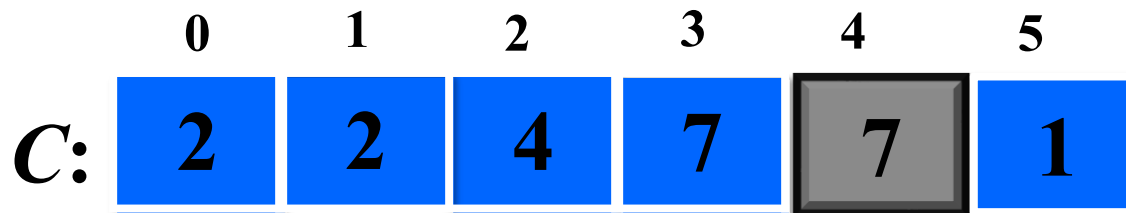
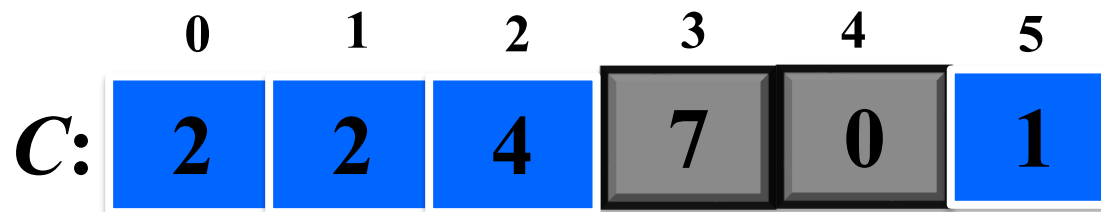
```
7. for i=1 to k  
8.   C[i] = C[i] + C[i-1];
```



Executing Loop 3

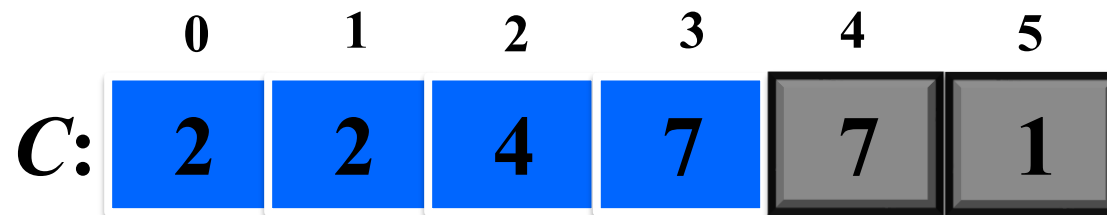
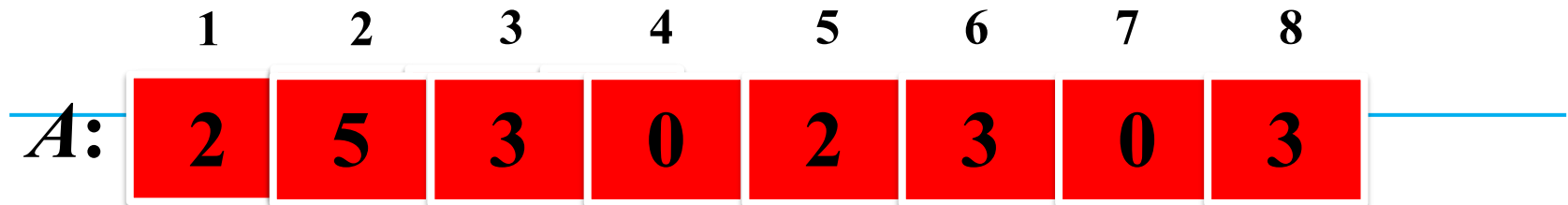


7. for i=1 to k
8. $C[i] = C[i] + C[i-1];$

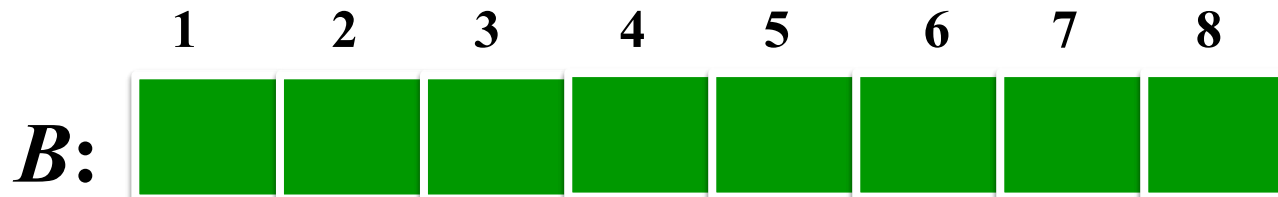
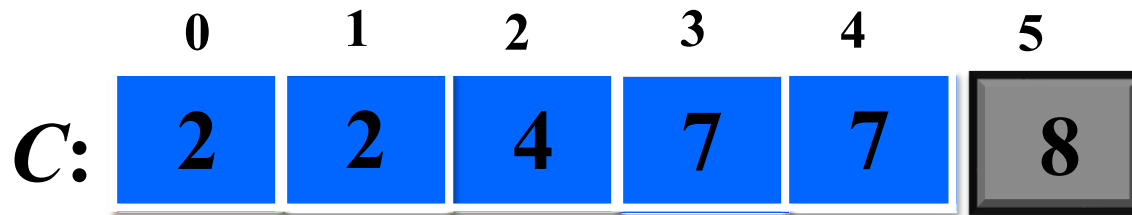




Executing Loop 3



7. for i=1 to k
8. $C[i] = C[i] + C[i-1];$





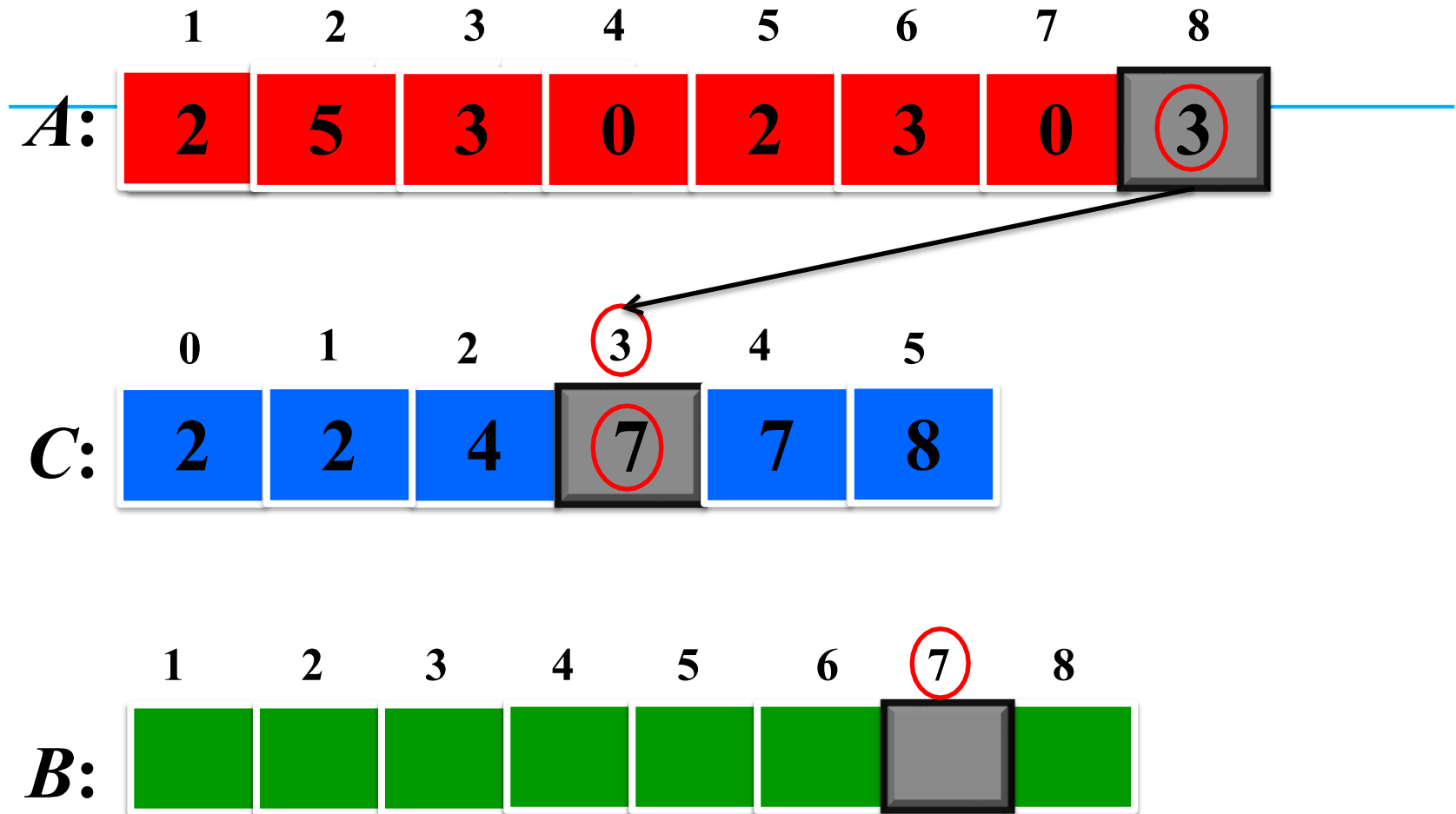
End of Loop 3

	1	2	3	4	5	6	7	8
<i>A:</i>	2	5	3	0	2	3	0	3

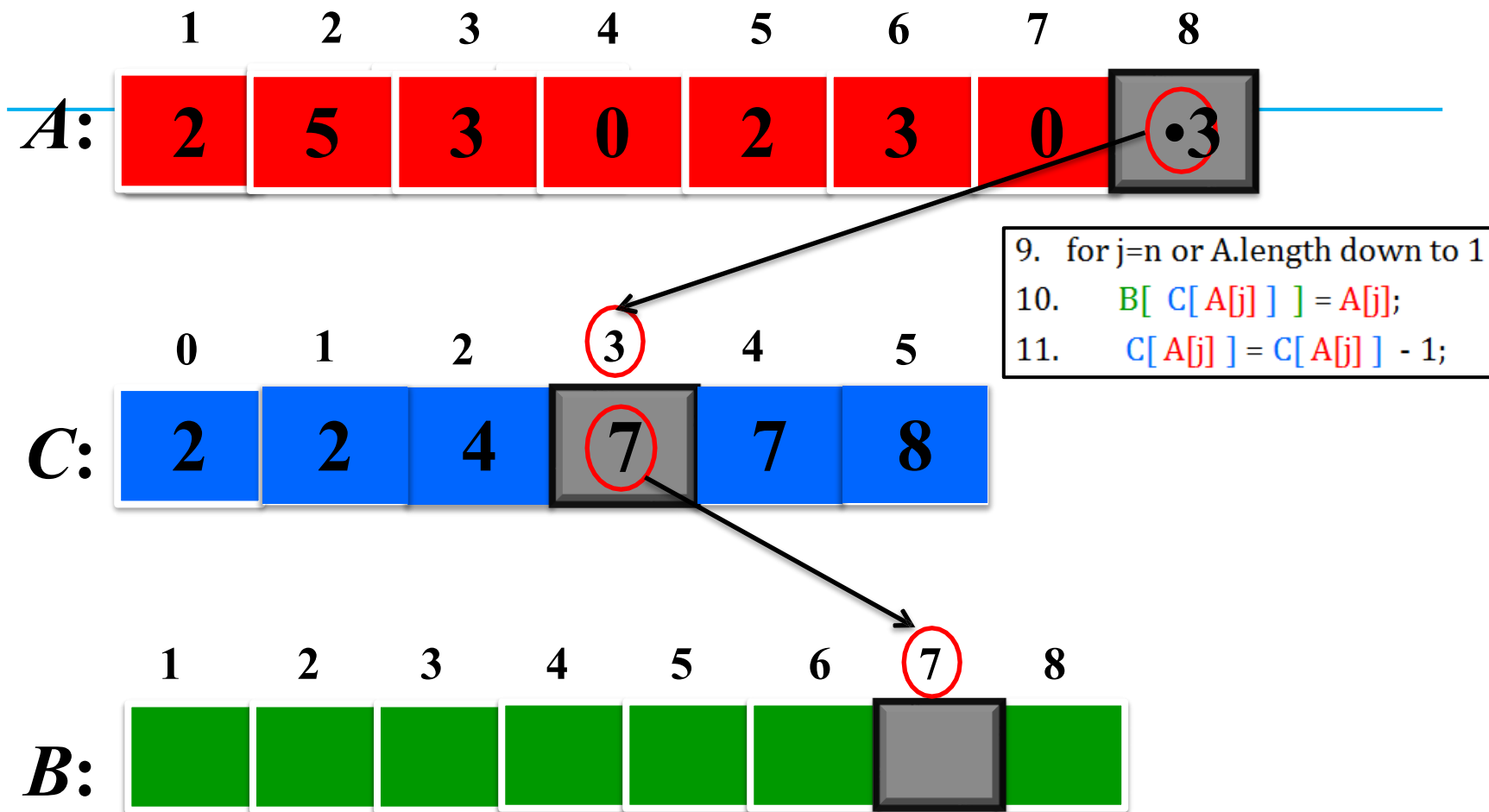
	0	1	2	3	4	5
<i>C:</i>	2	2	4	7	7	8

	1	2	3	4	5	6	7	8
<i>B:</i>								

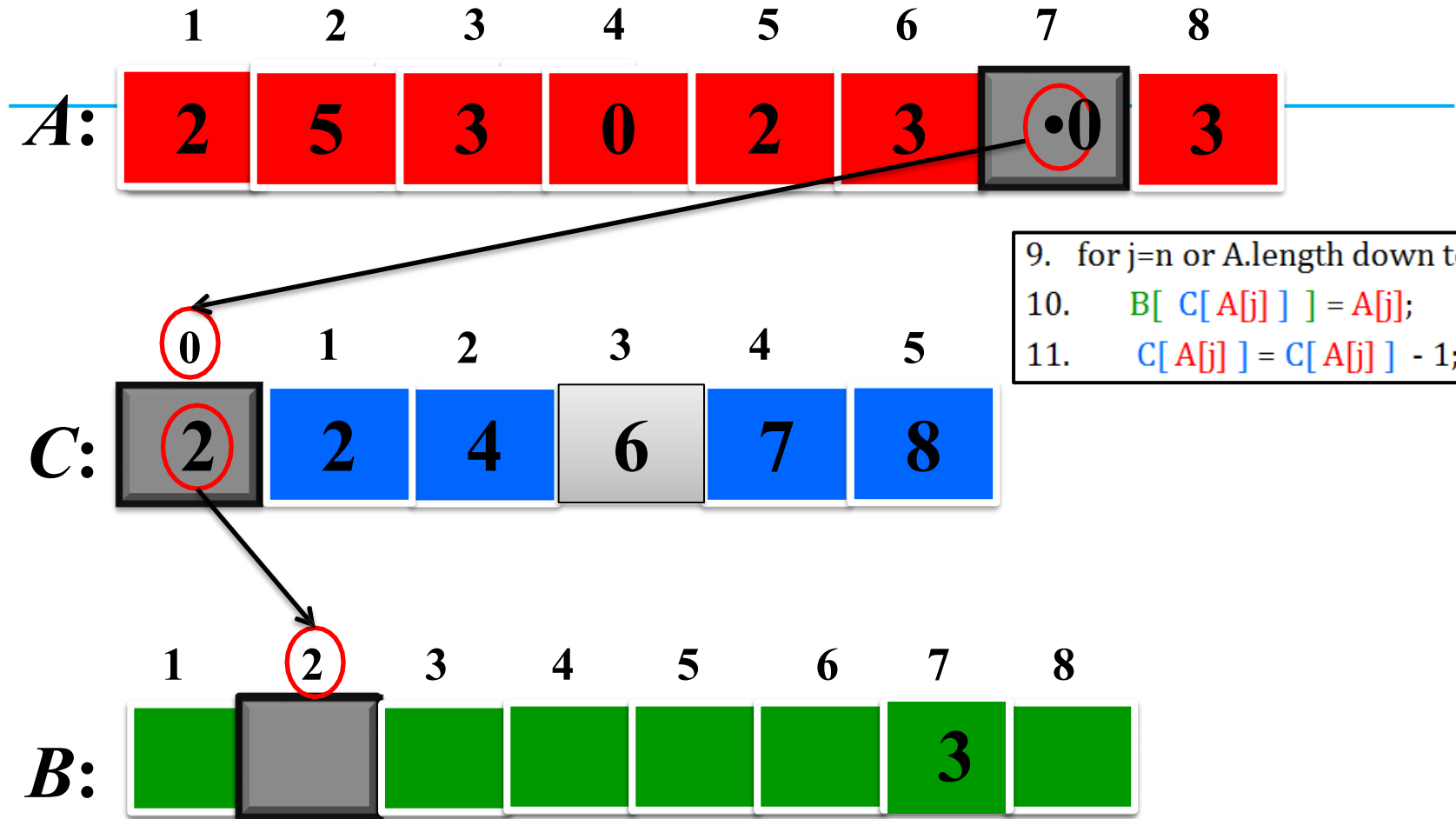
Executing Loop 4



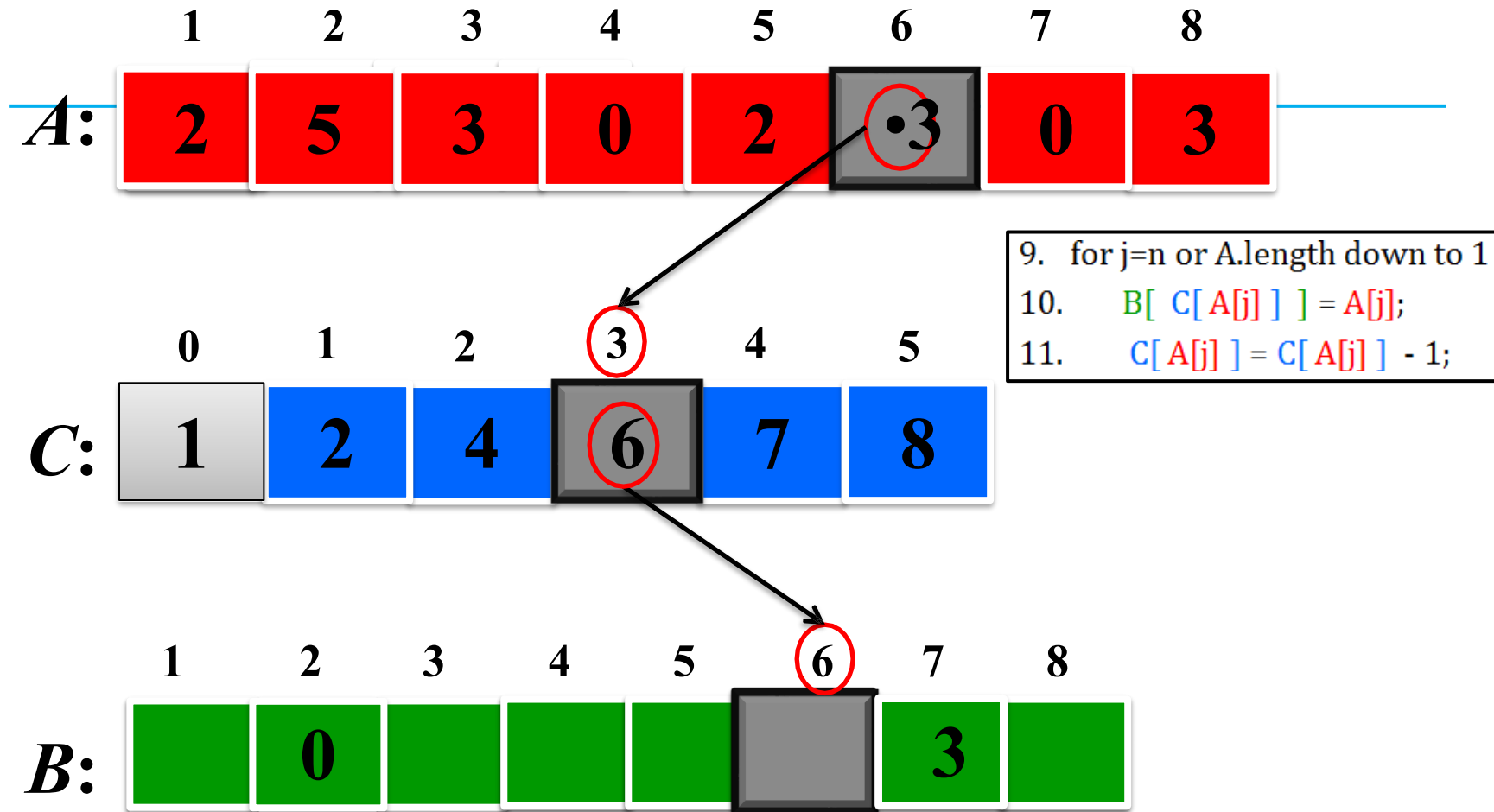
Executing Loop 4



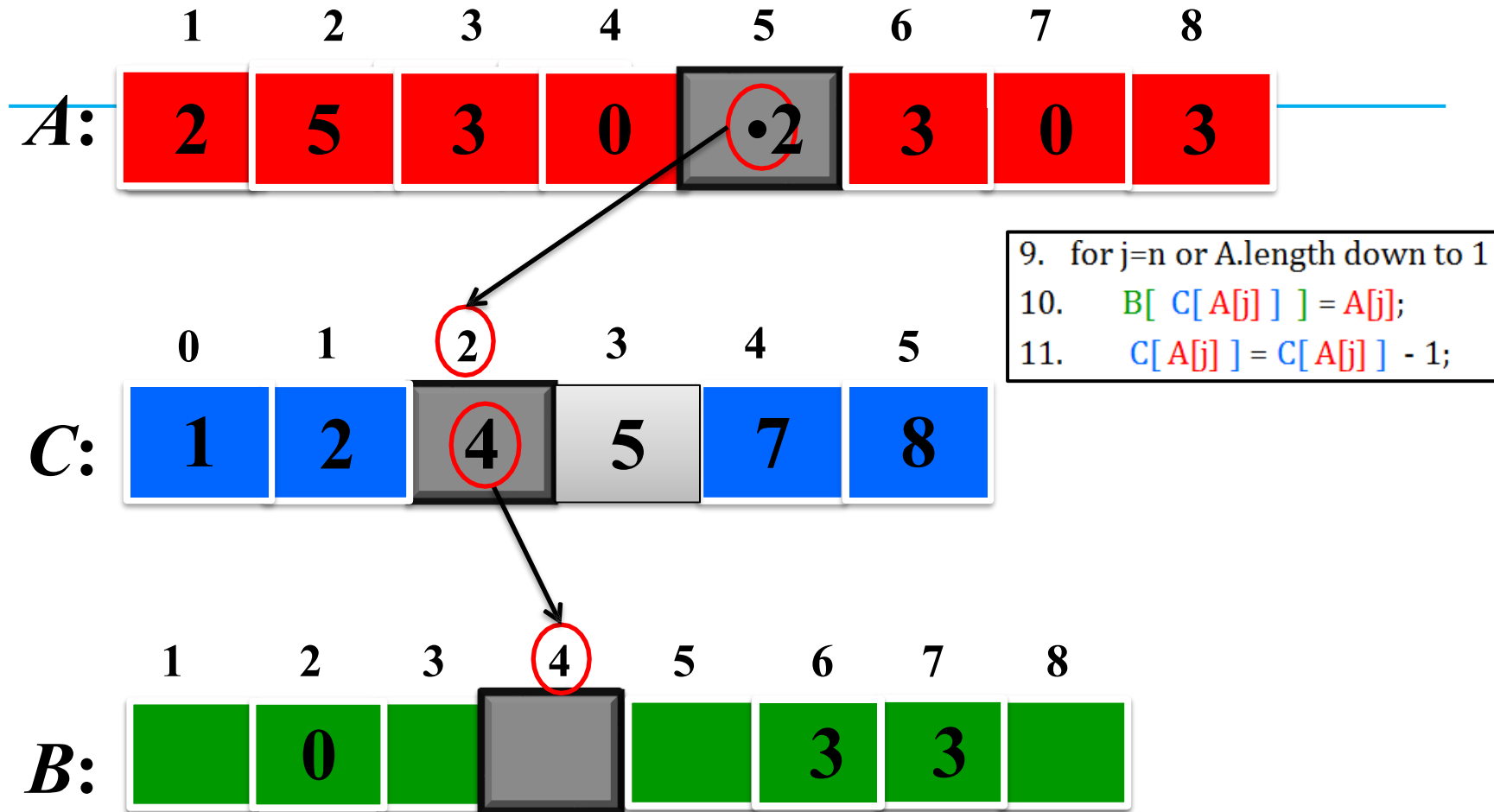
Executing Loop 4



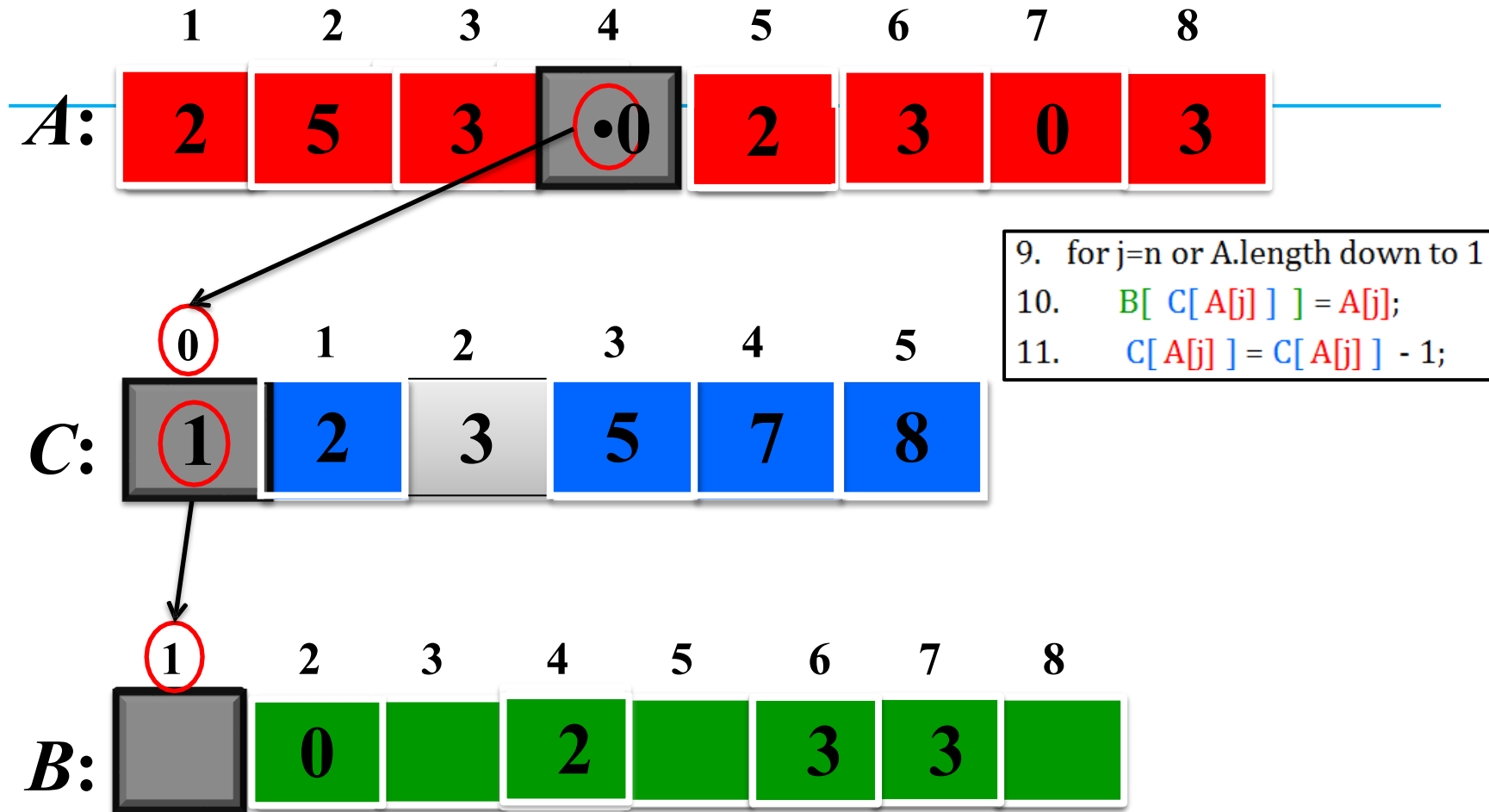
Executing Loop 4



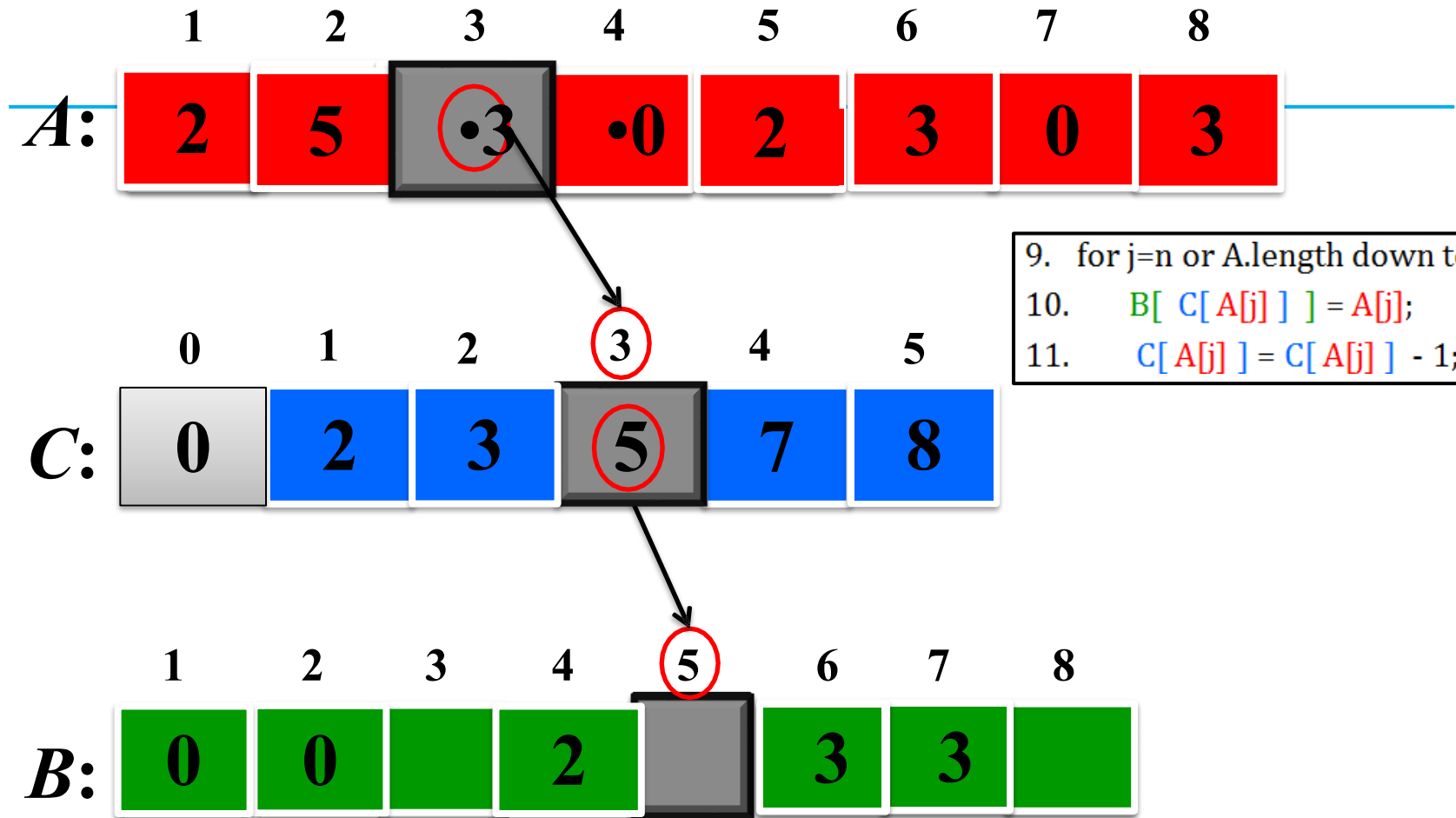
Executing Loop 4



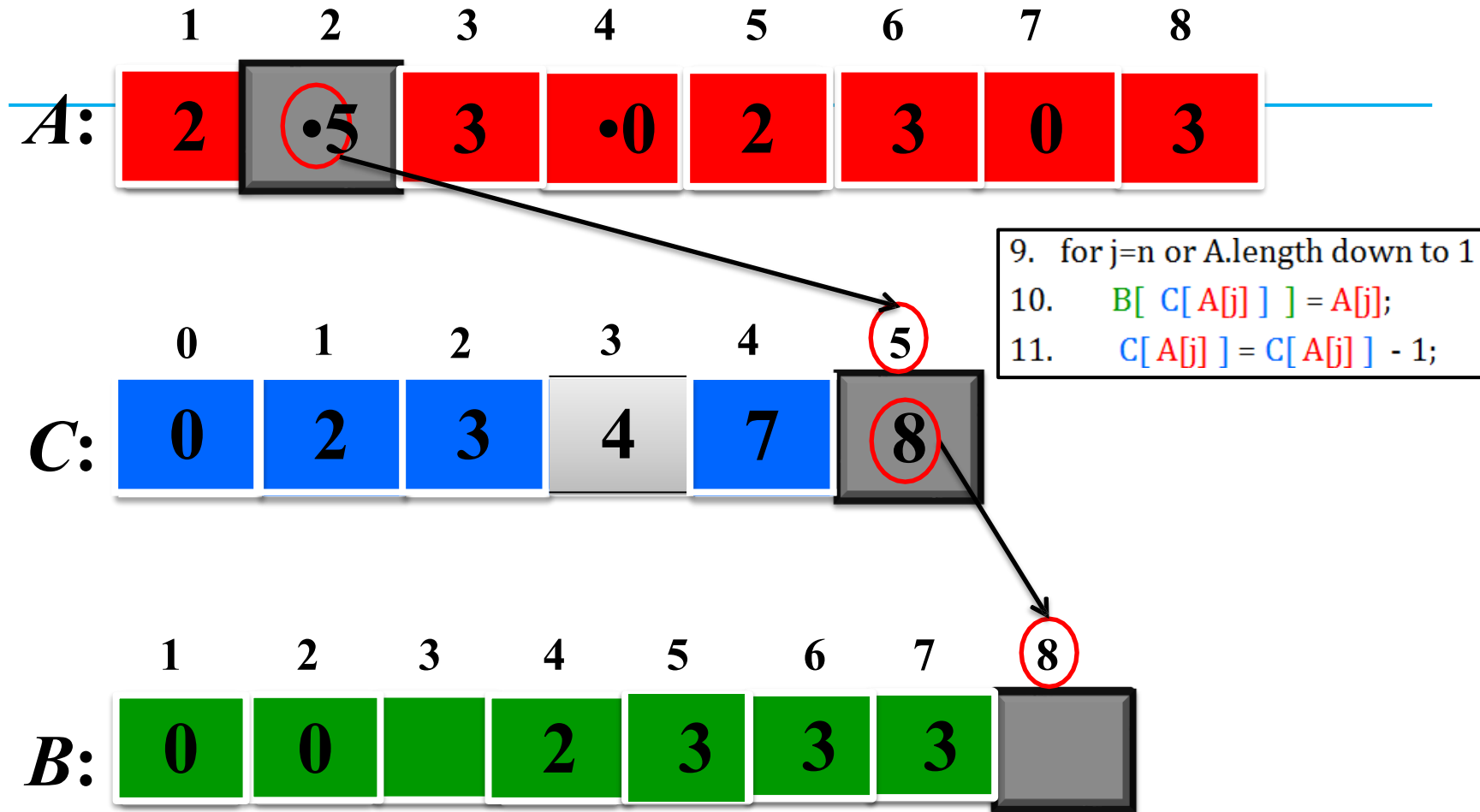
Executing Loop 4



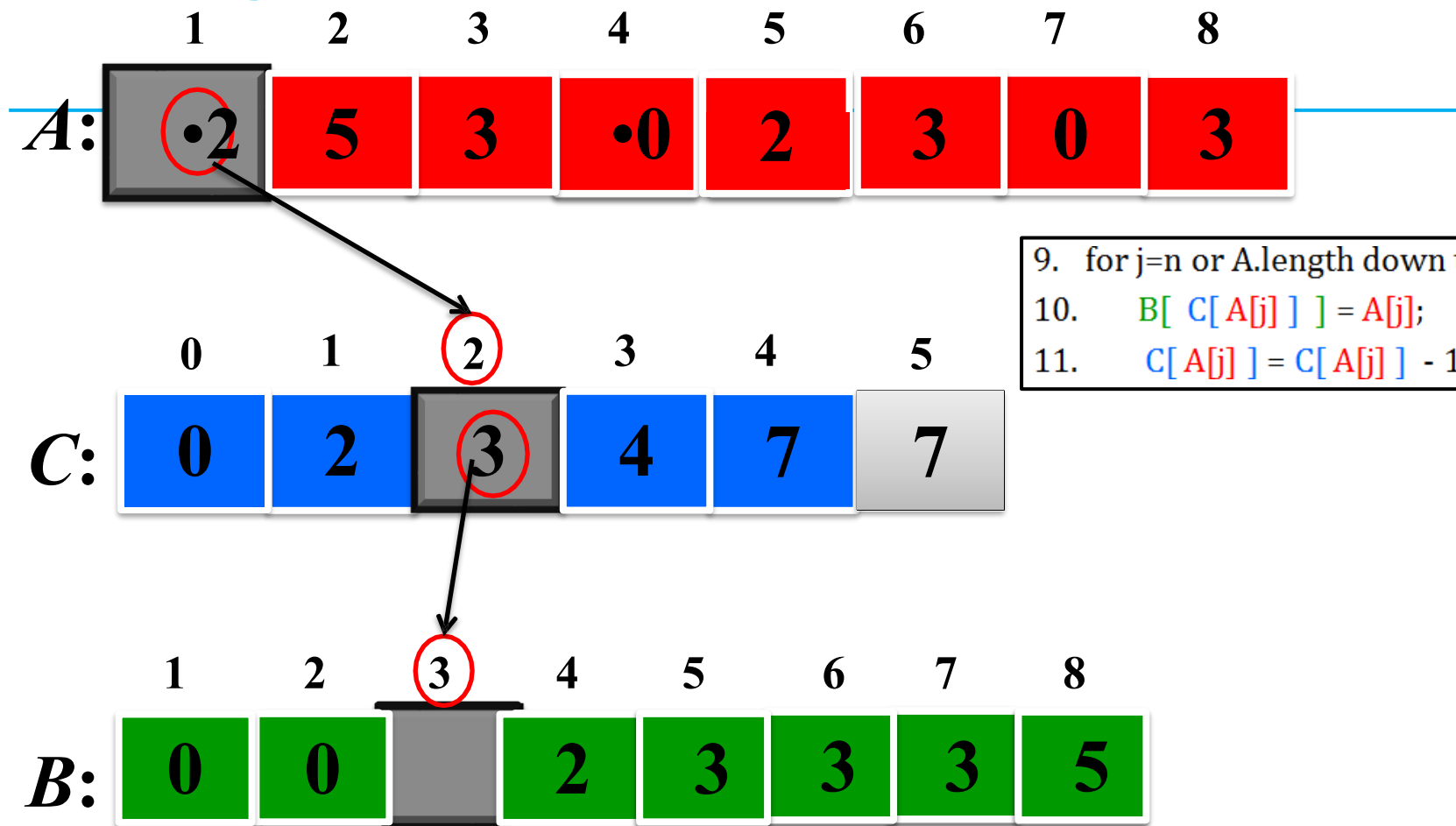
Executing Loop 4



Executing Loop 4



Executing Loop 4





End of Loop 4

	1	2	3	4	5	6	7	8
A:	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C:	0	2	2	4	7	7

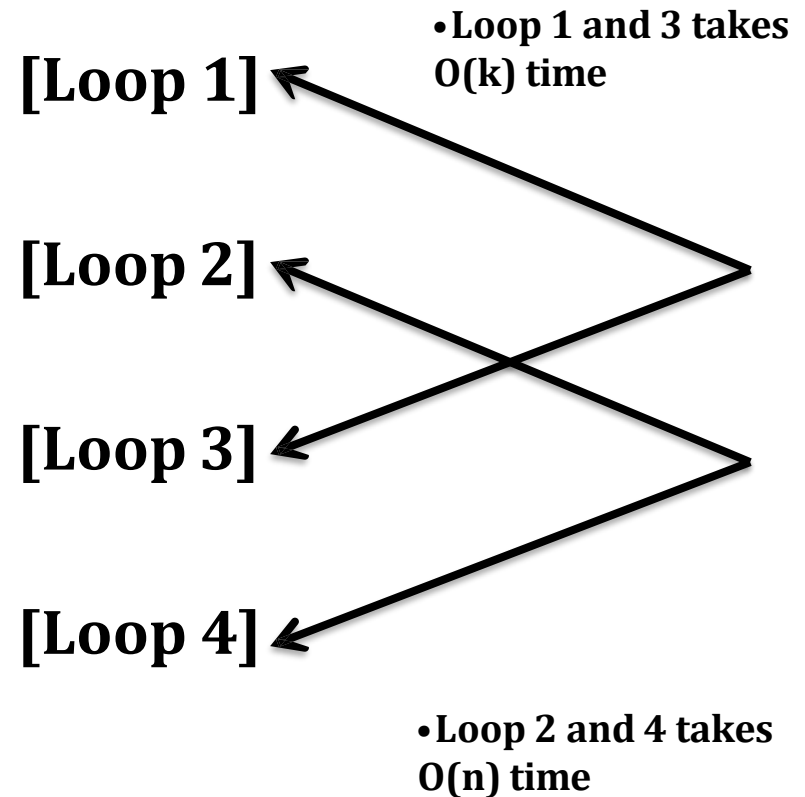
	1	2	3	4	5	6	7	8
B:	0	0	2	2	3	3	3	5

•Sorted data in Array B



Time Complexity Analysis

1. Counting-Sort(A, B, k)
2. Let $C[0 \dots k]$ be a new array
3. for $i=0$ to k
4. $C[i] = 0;$
5. for $j=1$ to A.length or n
6. $C[A[j]] = C[A[j]] + 1;$
7. for $i=1$ to k
8. $C[i] = C[i] + C[i-1];$
9. for $j=n$ or A.length down to 1
10. $B[C[A[j]]] = A[j];$
11. $C[A[j]] = C[A[j]] - 1;$





Complexity

Time Complexity	
Best	$O(n+\max)$
Worst	$O(n+\max)$
Average	$O(n+\max)$
Space Complexity	
	$O(\max)$
Stability	
	Yes



Counting Sort Applications

- **Grading Systems:** Efficient for sorting exam scores where marks are in a small range (e.g., 0–100).
- **Character or Letter Sorting:** Useful in sorting characters (based on ASCII values) for string processing tasks.
- **Used in Radix Sort:** Counting Sort is a key building block for Radix Sort, which is used for sorting large numbers and strings efficiently.
- **Frequency Counting:** Ideal for tasks like counting how many times each number or character appears in a dataset.
- **Image Processing:** Applied to sort or count pixel intensity values (0–255), useful in histogram equalization.



Conditions for Using Counting Sort

- **Integer Values Only:** Works only with integers, as each value maps directly to an array index.
- **Non-Negative Values:** Usually limited to non-negative numbers since negative indices aren't valid in arrays.
- **Limited Range of Values:** Efficient only when the range of values k isn't much larger than the number of items n , or it wastes space and time.

Count sort in Descending order





Reference and Important links

- <https://www.youtube.com/watch?v=imqr13aIBAY>
- <https://www.programiz.com/dsa/counting-sort>
- <https://www.cs.usfca.edu/~galles/visualization/CountingSort.html>